Reporte Técnico de Actividades Práctico-Experimentales Nro. 002

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Fernando Patiño Francis Valdiviezo Jimmy Ontaneda Ana Panamito Daniel Saavedra
Asignatura	Sistemas Digitales
Ciclo	5to
Unidad	1 – Microcontroladores para sistemas embebidos
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Juego de luces interactivo con ESP32
Nombre del Docente	Luis Darío Sinche C.
Fecha	23 de octubre de 2026
Horario	Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (simulación en Wokwi y prueba en la placa ESP32 del grupo)
Tiempo planificado en el Sílabo	3 horas (APE, semana 2): 2 h presenciales y 1 h para terminar el informe

2. Objetivo(s) de la Práctica

Objetivo. Programar en el ESP32 un juego de luces con cuatro LEDs cuya velocidad cambia con un pulsador, usando temporización no bloqueante con millis() y la lectura del pulsador con pull-up interna, antirrebote y detección de flanco.

3. Materiales, Reactivos, Equipos y Herramientas

- 1 ESP32 DevKit V1 de 30 pines + cable USB de datos (no solo de carga)
- 1 Protoboard

- 4 LEDs de 5 mm rojos, amarillos o verdes (no azules ni blancos: casi no encienden a 3,3 V)
- 4 Resistencias de 330 Ω
- 1 Pulsador de 4 patas
- 6 Cables jumper macho-macho

4. Procedimiento / Metodología Ejecutada

Parte A

- Accedimos a la página wokwi. com y comenzamos un nuevo proyecto utilizando la tarjeta ESP32.
- Actualizamos el archivo diagram. json al copiar la estructura predeterminada del circuito (Anexo A) en la que ya están conectados los componentes.
- Comprobamos que las conexiones coinciden con la tabla anterior, garantizando que los pines RX2, TX2, D18, D19 y D21 están correctamente asignados.
- En la sección sketch. ino, introdujimos el Código 1 (una secuencia simple sin delay()) y pusimos en marcha la simulación para observar la secuencia secuencial (16 → 17 → 18 → 19 → 16) cada $0,3s$.
- Modificamos la variable PERIODO a 1000ms y 100ms para evaluar su comportamiento, anotando las observaciones pertinentes.
- Regresamos el valor de PERIODO a 300ms y registramos evidencia visual del circuito en funcionamiento.

Parte B

- Sustituimos el script en sketch.ino el Código 2, que permite modificar la velocidad a través de toques interactivos (Lento: 400 ms a Medio: 150 ms a Rápido: 60 ms).
- Verificamos el cambio de velocidad en el Monitor Serie al presionar el botón.
- Observamos el comportamiento al mantener pulsado por tres segundos, notando que la velocidad se ajusta solo una vez.
- Para analizar el impacto del rebote, configuramos T_REBOTE en 0, realizamos 10 pulsaciones y contabilizamos el número de impresiones en el Monitor Serie.
- Luego, restablecimos T_REBOTE a 30, volvimos a hacer la prueba de las 10 pulsaciones y capturamos una imagen del Monitor Serie como prueba.

Parte C

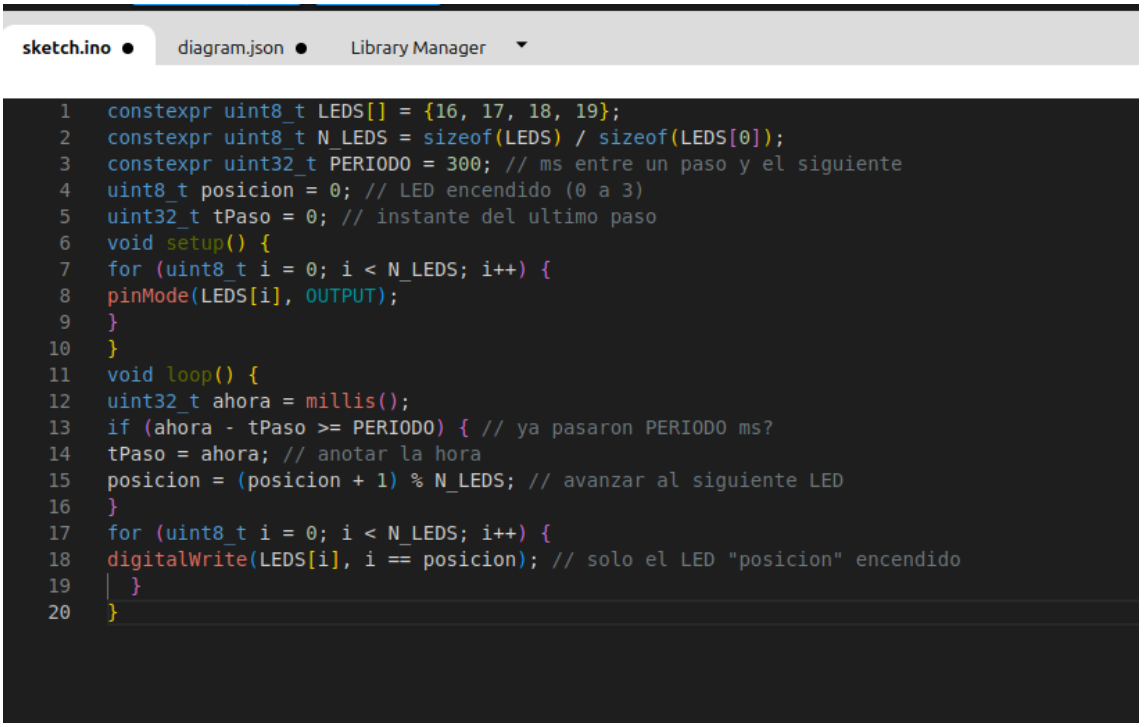
- Sin el cable USB conectado, procedemos a armar el circuito en la protoboard asegurándonos de seguir la correcta disposición de polaridad de los LEDs y las conexiones especificadas.
- A continuación, enlazamos la placa a la computadora a través de USB y preparamos el entorno en el Arduino IDE (opciones → Placa → Módulo ESP32 y seleccionamos el puerto COM/tty correspondiente).
- Luego, transferimos el Código 2 a la tarjeta (usando el botón BOOT si hay alguna interferencia al cargar). Iniciamos el Monitor Serie a una velocidad

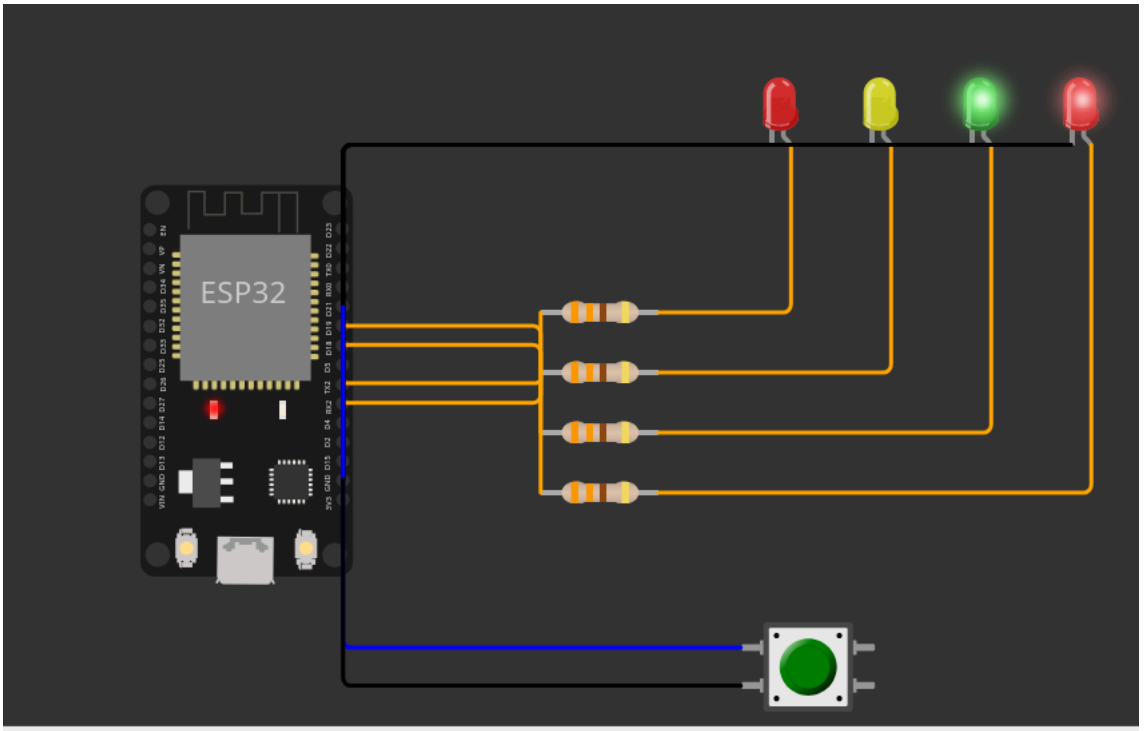
de 115200 baudios y comprobamos el cambio adecuado de velocidades utilizando el botón físico.

- Finalmente, grabamos un breve video (con una duración máxima de 30 segundos) que demuestra el correcto funcionamiento del hardware.

5. Resultados

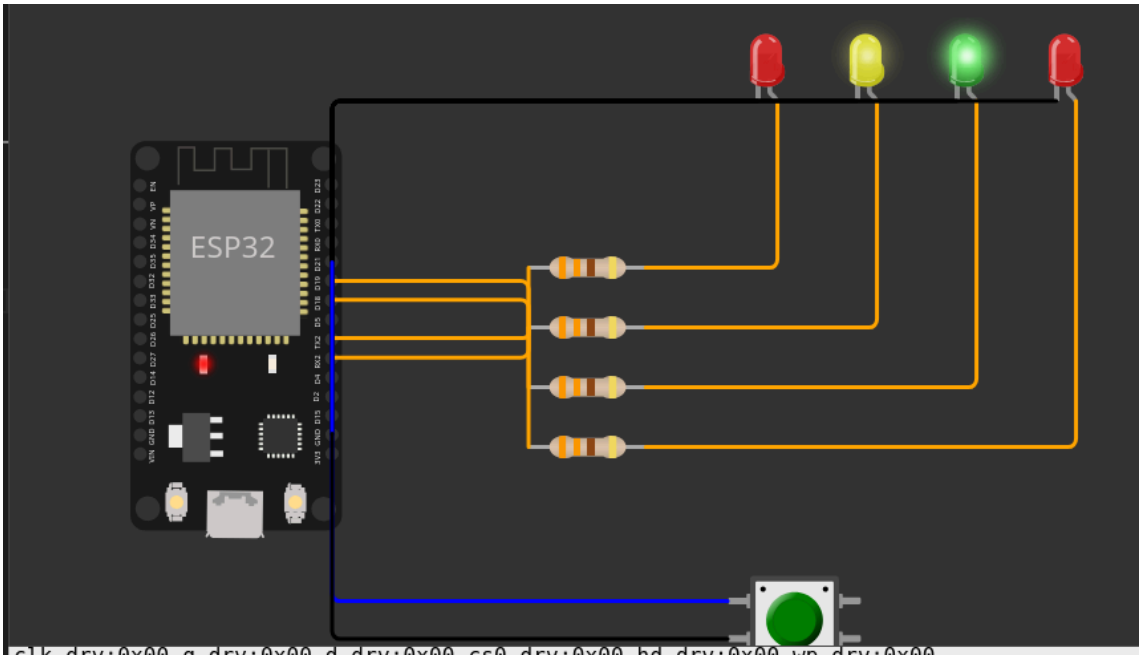
Código 1: un solo LED encendido a la vez, que avanza 16 → 17 → 18 → 19 → 16... cada 0,3 s; con PERIODO = 1000 y 100 la secuencia se vuelve más lenta o más rápida (Tabla A).






```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
```

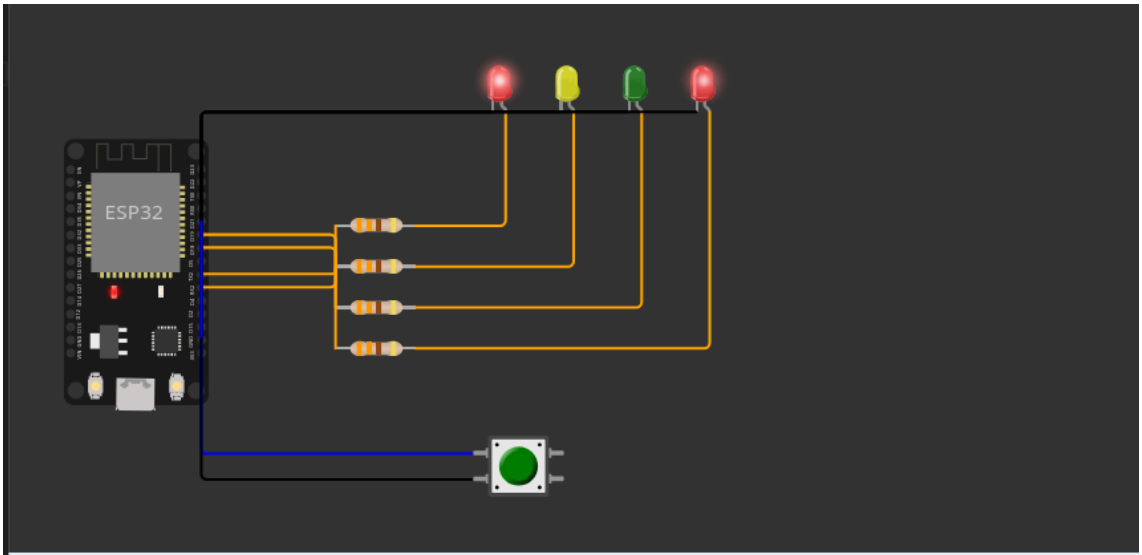
CON 1000




```
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
```

```
constexpr uint8_t LEDS[] = {16, 17, 18, 19};
constexpr uint8_t N_LEDS = sizeof(LEDS) / sizeof(LEDS[0]);
constexpr uint32_t PERIODO = 1000; // ms entre un paso y el s
uint8_t posicion = 0; // LED encendido (0 a 3)
uint32_t tPaso = 0; // instante del ultimo paso
void setup() {
for (uint8_t i = 0; i < N_LEDS; i++) {
pinMode(LEDS[i], OUTPUT);
}
}
void loop() {
uint32_t ahora = millis();
if (ahora - tPaso >= PERIODO) {
tPaso = ahora; // anotar la hora
posicion = (posicion + 1) % N_LEDS;
}
for (uint8_t i = 0; i < N_LEDS; i++) {
digitalWrite(LEDS[i], i == posicion);
  }
}
```

Con 100

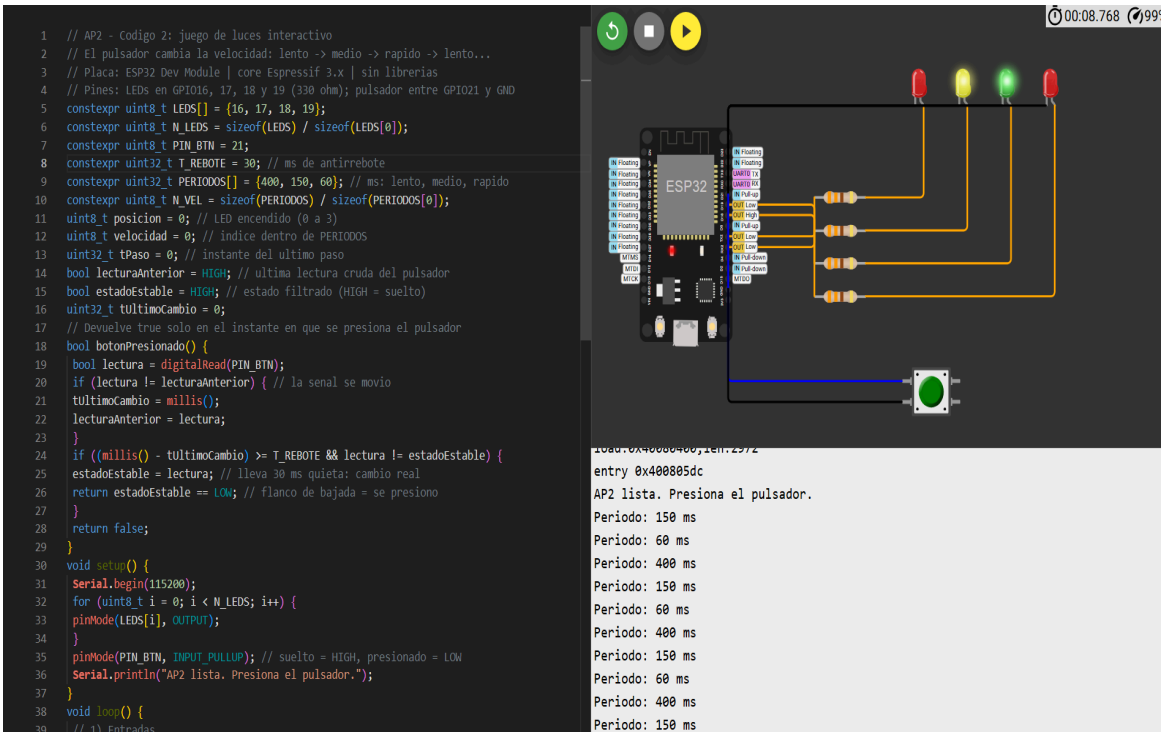



```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
```

```
1   constexpr uint8_t LEDS[] = {16, 17, 18, 19};
2   constexpr uint8_t N_LEDS = sizeof(LEDS) / sizeof(LEDS[0]);
3   constexpr uint32_t PERIODO = 100; // ms entre un paso y el si
4   uint8_t posicion = 0; // LED encendido (0 a 3)
5   uint32_t tPaso = 0; // instante del ultimo paso
6   void setup() {
7   for (uint8_t i = 0; i < N_LEDS; i++) {
8   pinMode(LEDS[i], OUTPUT);
9   }
10  }
11  void loop() {
12  uint32_t ahora = millis();
13  if (ahora - tPaso >= PERIODO) {
14  tPaso = ahora; // anotar la hora
15  posicion = (posicion + 1) % N_LEDS;
16  }
17  for (uint8_t i = 0; i < N_LEDS; i++) {
18  digitalWrite(LEDS[i], i == posicion);
19    }
20  }
```

https://wokwi.com/projects/478691381162414081

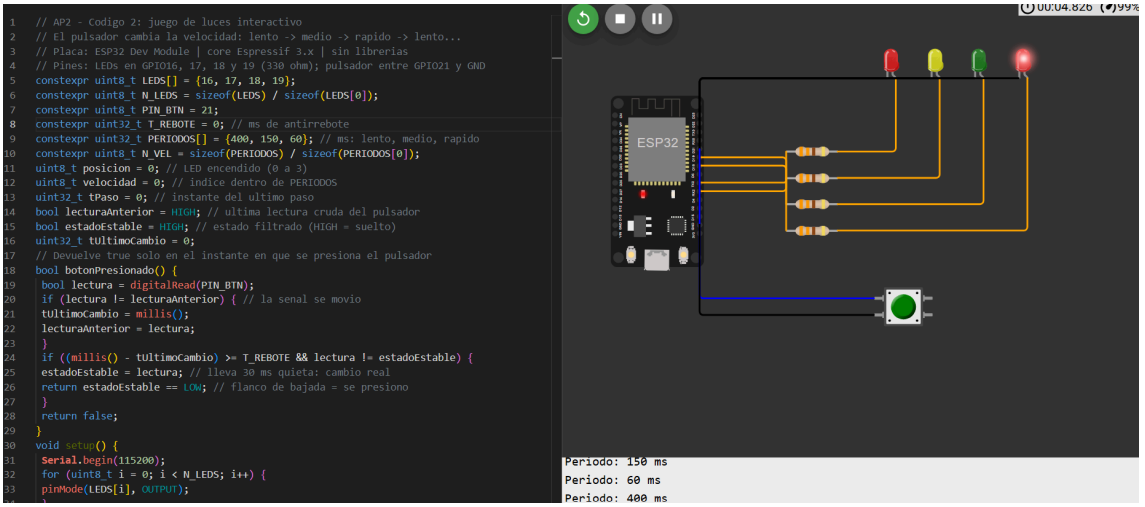
Código 2: cada clic cambia la velocidad (lento 400 ms → medio 150 ms → rápido 60 ms → lento...) y el Monitor Serie imprime Periodo: ... ms una sola vez por clic.



```
// AP2 - Codigo 2: juego de luces interactivo
// El pulsador cambia la velocidad: lento -> medio -> rapido -> lento...
// Placa: ESP32 Dev Module | core Espressif 3.x | sin librerias
// Pines: LEDs en GPIO16, 17, 18 y 19 (330 ohm); pulsador entre GPIO21 y GND
constexpr uint8_t LEDS[] = {16, 17, 18, 19};
constexpr uint8_t N_LEDS = sizeof(LEDS) / sizeof(LEDS[0]);
constexpr uint8_t PIN_BTN = 21;
constexpr uint32_t T_REBOTE = 30; // ms de antirrebote
constexpr uint32_t PERIODOS[] = {400, 150, 60}; // ms: lento, medio, rapido
constexpr uint8_t N_VEL = sizeof(PERIODOS) / sizeof(PERIODOS[0]);
uint8_t posicion = 0; // LED encendido (0 a 3)
uint8_t velocidad = 0; // indice dentro de PERIODOS
uint32_t tPaso = 0; // instante del ultimo paso
bool lecturaAnterior = HIGH; // ultima lectura cruda del pulsador
bool estadoEstable = HIGH; // estado filtrado (HIGH = suelto)
uint32_t tUltimoCambio = 0;
// Devuelve true solo en el instante en que se presiona el pulsador
bool botonPresionado() {
 bool lectura = digitalRead(PIN_BTN);
 if (lectura != lecturaAnterior) { // la senal se movio
 tUltimoCambio = millis();
 lecturaAnterior = lectura;
 }
 if ((millis() - tUltimoCambio) >= T_REBOTE && lectura != estadoEstable) {
 estadoEstable = lectura; // lleva 30 ms quieta: cambio real
 return estadoEstable == LOW; // flanco de bajada = se presiono
 }
 return false;
}
void setup() {
 Serial.begin(115200);
 for (uint8_t i = 0; i < N_LEDS; i++) {
 pinMode(LEDS[i], OUTPUT);
 }
 pinMode(PIN_BTN, INPUT_PULLUP); // suelto = HIGH, presionado = LOW
 Serial.println("AP2 lista. Presiona el pulsador.");
}
void loop() {
```

```
entry 0x400805dc
AP2 lista. Presiona el pulsador.
Periodo: 150 ms
Periodo: 60 ms
Periodo: 400 ms
Periodo: 150 ms
Periodo: 60 ms
Periodo: 400 ms
Periodo: 150 ms
Periodo: 60 ms
Periodo: 400 ms
Periodo: 150 ms
```

Clic sostenido: la velocidad cambia una sola vez, aunque el botón siga presionado.



```
// AP2 - Codigo 2: juego de luces interactivo
// El pulsador cambia la velocidad: lento -> medio -> rapido -> lento...
// Placa: ESP32 Dev Module | core Espressif 3.x | sin librerias
// Pines: LEDs en GPIO16, 17, 18 y 19 (330 ohm); pulsador entre GPIO21 y GND
constexpr uint8_t LEDS[] = {16, 17, 18, 19};
constexpr uint8_t N_LEDS = sizeof(LEDS) / sizeof(LEDS[0]);
constexpr uint8_t PIN_BTN = 21;
constexpr uint32_t T_REBOTE = 0; // ms de antirrebote
constexpr uint32_t PERIODOS[] = {400, 150, 60}; // ms: lento, medio, rapido
constexpr uint8_t N_VEL = sizeof(PERIODOS) / sizeof(PERIODOS[0]);
uint8_t posicion = 0; // LED encendido (0 a 3)
uint8_t velocidad = 0; // indice dentro de PERIODOS
uint32_t tPaso = 0; // instante del ultimo paso
bool lecturaAnterior = HIGH; // ultima lectura cruda del pulsador
bool estadoEstable = HIGH; // estado filtrado (HIGH = suelto)
uint32_t tUltimoCambio = 0;
// Devuelve true solo en el instante en que se presiona el pulsador
bool botonPresionado() {
 bool lectura = digitalRead(PIN_BTN);
 if (lectura != lecturaAnterior) { // la senal se movio
 tUltimoCambio = millis();
 lecturaAnterior = lectura;
 }
 if ((millis() - tUltimoCambio) >= T_REBOTE && lectura != estadoEstable) {
 estadoEstable = lectura; // lleva 30 ms quieta: cambio real
 return estadoEstable == LOW; // flanco de bajada = se presiono
 }
 return false;
}
void setup() {
 Serial.begin(115200);
 for (uint8_t i = 0; i < N_LEDS; i++) {
 pinMode(LEDS[i], OUTPUT);
```

```
Periodo: 150 ms
Periodo: 60 ms
Periodo: 400 ms
```

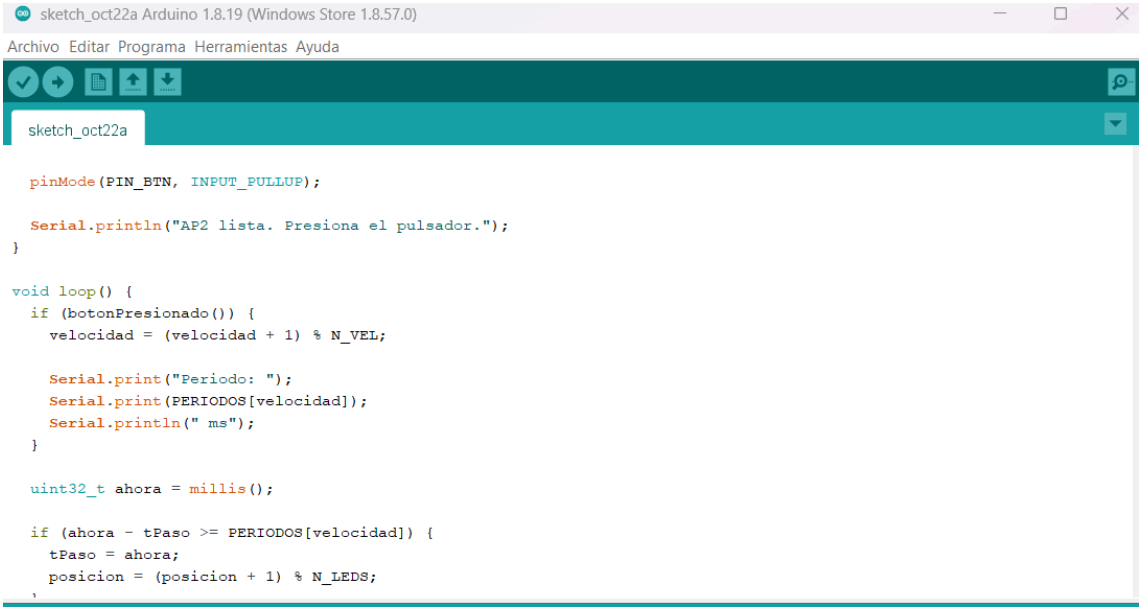
Link del Wokwi: https://wokwi.com/projects/478691329132057601

Placa real (Parte C): el mismo comportamiento que en Wokwi.

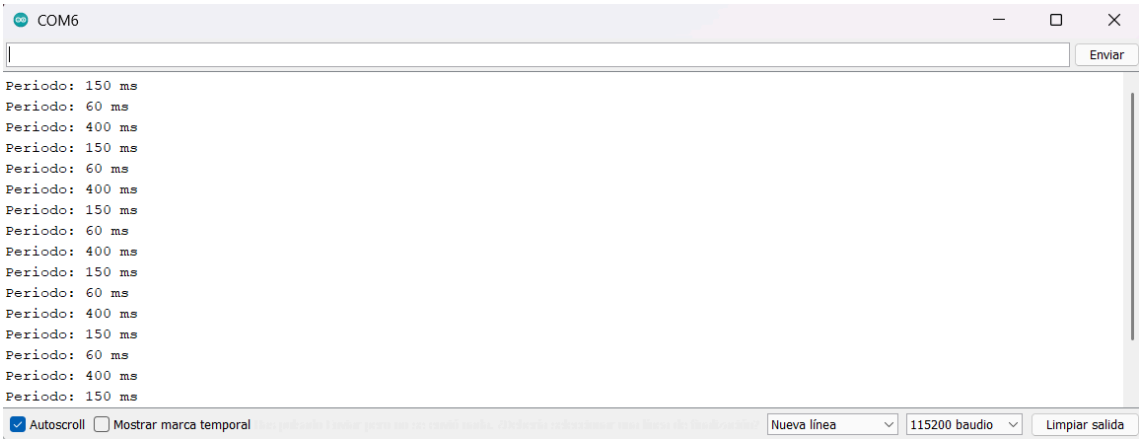
Sección C:

C2:

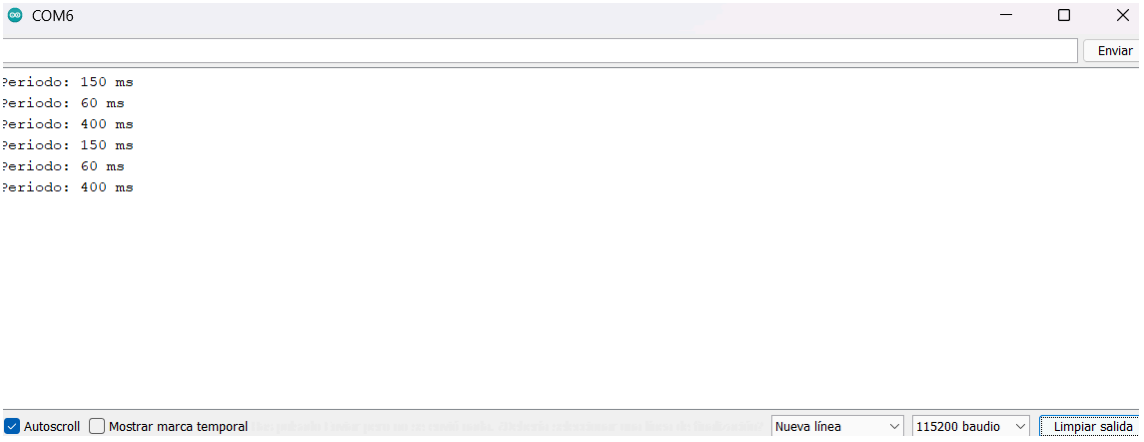
C3:



C4 - B2:



C4 - B3:



Se realizo la prueba aplastando 3s el boton, el resultado nos dio el mismo caso, no hubo casos rebote

Se debe entregar el esquema de conexiones, los códigos, las evidencias (capturas de Wokwi y, si hubo Parte C, el video) y las Tablas A y B completas.

Tabla A. Efecto del periodo

PERIODO(ms)	Que se observa?
1000	Que el cambio entre los leds es mas lento ya que es mayor el tiempo de cambio entre leds
300	Se puede decir que es una velocidad casi intermedia que el cambio no es tan rápido.
100	Vemos que la velocidad de cambio es rápida ya que hay menos tiempo de cambio y hace más rápido el cambio.

Tabla B. Efecto del antirrebote (10 clics)

T_REBOTE (ms)	Líneas impresas	¿Coincide con los 10 clics?
0	Periodo: 60 ms Periodo: 400 ms Periodo: 150 ms Periodo: 60 ms Periodo: 400 ms Periodo: 150 ms Periodo: 60 ms Periodo: 400 ms Periodo: 150 ms Periodo: 60 ms	SI
30	Periodo: 150 ms Periodo: 60 ms Periodo: 400 ms Periodo: 150 ms Periodo: 60 ms Periodo: 400 ms Periodo: 150 ms Periodo: 60 ms Periodo: 400 ms Periodo: 150 ms	SI

6. Preguntas de Control

1. **¿Por qué el Código 1 usa millis() en lugar de delay()? ¿Qué pasaría con el pulsador del Código 2 si cada paso de los LEDs usara delay(400)?**

   millis(): No bloquea la ejecución. Permite revisar el estado del pulsador en cada ciclo mientras cuenta el tiempo.

   delay(400): Congela el procesador durante 400 ms, ignorando las pulsaciones recibidas durante ese intervalo.

2. **El pulsador va entre el GPIO21 y GND, sin resistencia externa. ¿Qué valor lee digitalRead() con el pulsador suelto y con el pulsador presionado? ¿Por qué?**

   Suelto: HIGH (1).

   Presionado: LOW (0).

   La resistencia interna conecta el pin a 3,3 V en reposo. Al presionar, el botón deriva la señal directamente a GND (0 V).

3. **¿Qué pasaría si conectaran el pulsador al GPIO34 sin cambiar el código? Justifiquen con lo visto en clase.**

   Lecturas erráticas o falla total.

   El GPIO34 es un pin exclusivamente de entrada (*Input-Only*) que no posee resistencia interna de pull-up. Sin una resistencia física externa, la lectura queda flotante (*floating*).

4. **En el paso B3 la velocidad cambió una sola vez aunque mantuvieron presionado el pulsador. ¿Qué parte del Código 2 lo garantiza?**

   Garantía: La detección de flanco de bajada (*falling edge*).

   Mecanismo: Compara el estado actual con el anterior (estadoAnterior == HIGH && estadoActual == LOW), activándose solo en el instante preciso en que el botón pasa de suelto a presionado.

5. **Expliquen la Tabla B: ¿por qué con T_REBOTE = 0 puede haber más líneas impresas que clics?**

   El rebote mecánico (*contact bounce*). Los contactos internos del botón vibran al unirse, generando múltiples micro-pulsos en pocos milisegundos. Sin el filtro por software, el programa registra cada vibración como un clic independiente.

6. **Calcule la corriente que circula por un LED rojo (Vf ≈ 2,0 V) con una resistencia de 330 Ω a 3,3 V. ¿Por qué no conviene usar un LED azul?**

   Datos:

   - Voltaje de alimentacion (Vcc) = 3.3 V
   - Voltaje directo del LED (Vf) = 2.0 V
   - Resistencia (R) = 330 ohmios

   Formula: I = (Vcc - Vf) / R

   Voltaje en la resistencia (Vr) = 3.3 V - 2.0 V = 1.3 V

   Corriente (I) = 1.3 V / 330 ohmios

   I = 0.003939 A = 3.94 mA

   La corriente que circula por el LED rojo es aproximadamente 3.94 mA.

7. **Mencionen una diferencia entre probar el circuito en Wokwi y probarlo en la placa real.**

Wokwi modela componentes ideales (sin rebote mecánico ni ruido eléctrico). La placa real sufre interferencias de la protoboard, tolerancias de componentes y rebotes reales en el pulsador.

8. **El ESP32 sigue funcionando a 240 MHz aunque el juego solo cambie de LED cada 400 ms. Propongan una idea para reducir el consumo de energía si el juego funcionara con batería.**

   Reducir la frecuencia de reloj: Bajar el CPU de 240 MHz a 20 u 80 MHz con setCpuFrequencyMhz(20).

   Modos de suspensión: Poner el ESP32 en *Light Sleep* entre cada cambio de LED y despertarlo por temporizador o interrupción por el pulsador.

7. Conclusiones

- Se logró programar un juego de luces con cuatro LEDs en el ESP32, cuya velocidad cambia mediante un pulsador. El uso de `millis()` permitió controlar la secuencia sin bloquear la ejecución, mientras que la resistencia pull-up interna, el antirrebote y la detección de flanco permitieron registrar cada pulsación de forma estable, evitando cambios involuntarios de velocidad.
- El desarrollo de la práctica evidenció la importancia de conocer las características físicas del ESP32, tales como la ausencia de resistencias *pull-up* en pines únicamente de entrada (como el GPIO34) y las caídas de tensión según el color del LED. Asimismo, se concluyó que si bien Wokwi facilita la verificación lógica en un entorno ideal, la placa real exige considerar el ruido eléctrico y las tolerancias mecánicas reales de los componentes.
- Se identificó que ejecutar secuencias temporizadas sencillas a la frecuencia por defecto del ESP32 (240 MHz) genera un consumo innecesario de energía. A partir de esto, se concluyó la viabilidad de aplicar técnicas de eficiencia energética —como el escalado dinámico de la frecuencia del CPU o el uso de modos de suspensión (*Light Sleep*)— para extender la autonomía del sistema en aplicaciones portátiles alimentadas por batería.

8. Declaración de IA

Se utilizó una herramienta de Inteligencia Artificial (Gemini)como asistente de redacción para sintetizar y estructurar adecuadamente las conclusiones a partir de los datos y resultados experimentales obtenidos en el laboratorio. Toda la información, análisis y conceptos presentados fueron revisados, validados y aprobados íntegramente por los integrantes del grupo de trabajo.

9. Anexos

Link de repositorio:
https://github.com/Dan-San837/AP2-SistemasDigitales-Saavedra-Pati-o-Valdiviezo-Ontaneda-Panamito.git